Four Snacks, Line by Line

C#, WPF, SQL Server and plugins: the Genetec projects explained

A walkthrough of the four Genetec snacks: what each one does, why it is built the way it is, and the code that matters, explained line by line, with the tests that check it. Interview questions and exercises close every part.

Part	What it covers
0. The big picture	How the snacks connect, the shared build, how to read the book
1. Camera Status Monitor	WPF and MVVM, commands, the UI thread, testing ViewModels
2. Access Event Log	SQL Server schema and indexes, Dapper, reports, contract tests, .NET Framework 4.8
3. Mini Plugin Framework	Reflection, AssemblyLoadContext, fault isolation, commands, configuration
4. Web Dashboard	ASP.NET Core minimal APIs, hosted services, React and TypeScript
5. Keeping it working	Lock files, Semantic Versioning, CI on Windows and Linux, Dependabot

Every code listing shows the real file and its real line numbers. The notes under a listing refer to those numbers, so you can read the book with the project open beside it.

Contents

Access Event Log 3

2.1 What it does 4

2.2 The schema 4

2.3 Repository, Dapper and contract tests 5

2.4 One library, two frameworks 5

2.5 The model 6

2.6 The repository interface 7

2.7 schema.sql 8

2.8 SQL Server with Dapper 9

2.9 The in-memory twin 12

2.10 The simulator 14

2.11 The ViewModel 15

2.12 The .NET Framework 4.8 app 17

2.13 The contract tests 19

2.14 Interview questions 21

2.15 Exercises 22

Part 2

Access Event Log

Snack 1's events, saved and searchable: a SQL Server schema with constraints and indexes, two reports written as real SQL, a repository with a Dapper implementation and an in-memory one held to the same contract, and a WPF app on .NET Framework 4.8.

2.1 What it does

Every badge swipe and every camera status change is recorded. The Events tab lists them newest first, with a search box (door, camera, location or cardholder) and an outcome filter (granted, denied, status changes). The Reports tab answers two questions an operator actually asks: *which doors denied access in the last 24 hours, and how often*, and *which five devices are the noisiest*. A simulator records a new event every 1.5 seconds, so both tabs move while you watch.

The app stores events in SQL Server when it can reach one, and falls back to an in-memory store when it can't, saying which in the status bar.

2.2 The schema

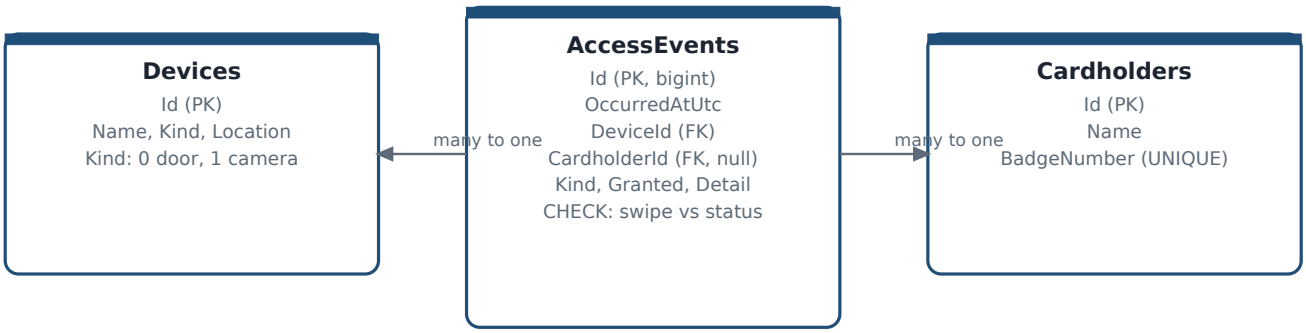


Figure 2.1. Three tables. An event always has a device; only badge swipes have a cardholder.

Constraints make bad data impossible rather than unlikely. Foreign keys stop an event pointing at a device that doesn't exist; a unique constraint stops two people having the same badge; a `CHECK` constraint states that a badge swipe has a cardholder and a decision while a status change has neither. The application checks too, but the database is the last line of defence, and it protects against every client, not just this one.

Indexes are the other half. Without one, "denied in the last 24 hours" reads every row ever recorded. An index is a sorted copy of some columns that lets SQL Server jump to the start of the time range and read only what's in it (a *seek* instead of a *scan*). If the index also *includes* the other columns the query needs, SQL Server never visits the table at all: a **covering index**.

2.3 Repository, Dapper and contract tests

A **repository** is an interface describing storage in the application's terms (record an event, search, run a report) rather than the database's. Code above it doesn't know where data lives, so the same ViewModel runs on SQL Server or in memory, and tests use the in-memory one.

Dapper is a micro-ORM: you write the SQL, it opens nothing for you, and it maps result columns onto object properties by name. That suits this snack, whose point is the SQL. Entity Framework would generate the SQL instead (and EF Core doesn't run on .NET Framework).

Two implementations of one interface can drift: the in-memory one might sort differently or forget a rule. **Contract tests** prevent that. The tests are written once, against the interface, in an abstract base class; one subclass runs them on the in-memory store, another on a real SQL Server. If they disagree, a test fails.

2.4 One library, two frameworks

MSBuild snack2-access-event-log/src/AccessLog.Core/AccessLog.Core.csproj, lines 1–30

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <!--
4      The model, the repository (SQL Server and in-memory), the simulator and the ViewModels.
5      net48 for the WPF app the README asks for; net10.0 for the tests and Snack 4's web API.
6    -->
7    <PropertyGroup>
8      <TargetFrameworks>net48;net10.0</TargetFrameworks>
9    </PropertyGroup>
10
11   <ItemGroup>
12     <PackageReference Include="Dapper" Version="2.1.89" />
13     <PackageReference Include="Microsoft.Data.SqlClient" Version="7.1.1" />
14     <!--
15       SqlClient depends on System.Security.Cryptography.Pkcs, and the versions it asks for have
16       known vulnerabilities. Referencing a patched version directly overrides the transitive one
         .
17     -->
18     <PackageReference Include="System.Security.Cryptography.Pkcs" Version="10.0.15" />
19   </ItemGroup>
20
21   <!-- Lets net48 build anywhere (including Linux CI) without the .NET Framework targeting pack.
          -->
22   <ItemGroup Condition="'$(TargetFramework)' == 'net48'">
23     <PackageReference Include="Microsoft.NETFramework.ReferenceAssemblies" Version="1.0.3" Priva
         teAssets="all" />
24   </ItemGroup>
25
26   <ItemGroup>
27     <EmbeddedResource Include="Data\schema.sql" />
28   </ItemGroup>
29
30 </Project>
```

Line 8 Plural `TargetFrameworks`: the project compiles twice. `net48` for the WPF app the README asks for; `net10.0` for the tests and for Snack 4's web API. The code may only use what both frameworks have, so there's no `string.Contains(string, StringComparison)` or `Lock` in this project.

Line 15 A **transitive dependency override**. SqlClient depends on Pkcs; the versions it asked for had known vulnerabilities, so CI's restore warned. Referencing a patched version directly makes NuGet pick it. Part 5 tells the story.

Line 23 The .NET Framework's reference assemblies as a package, so `net48` compiles on machines (and CI runners) without the targeting pack installed. `PrivateAssets="all"` keeps it out of anything that references this project.

Line 27 The schema ships inside the DLL, so the app can create its database wherever it runs, with no file to deploy alongside.

2.5 The model

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 11–19

```
11  /// <summary>What happened at a device. Stored as a number in AccessEvents.Kind.</summary>
12  public enum EventKind
13  {
14      /// <summary>A cardholder presented a badge. <see cref="AccessEvent.Granted"/> says what the
        door decided.</summary>
15      BadgeSwipe = 0,
16
17      /// <summary>A device went online, offline or into alarm (Snack 1's status changes).</summar
        y>
18      StatusChange = 1,
19  }
```

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 38–55

```
38  /// <summary>A row of the AccessEvents table. Times are always UTC.</summary>
39  public sealed class AccessEvent
40  {
41      public long Id { get; set; }
42      public DateTime OccurredAtUtc { get; set; }
43      public int DeviceId { get; set; }
44
45      /// <summary>Who swiped. Null for status changes.</summary>
46      public int? CardholderId { get; set; }
47
48      public EventKind Kind { get; set; }
49
50      /// <summary>Access granted (true) or denied (false). Null for status changes.</summary>
51      public bool? Granted { get; set; }
52
53      /// <summary>Free text, e.g. "Online → Alarm" for a status change.</summary>
54      public string? Detail { get; set; }
55  }
```

Plain classes with settable properties, not records: Dapper fills them by setting properties, and records with `init` setters need a compiler helper type that .NET Framework doesn't have. `Granted` is `bool?` because a status change has no decision: null means "not applicable", which a plain `bool` can't say.

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 57–78

```
57  /// <summary>An event joined with its device and cardholder, as the event list shows it.</summar
        y>
58  public sealed class AccessEventView
59  {
60      public long Id { get; set; }
```

```
61      public DateTime OccurredAtUtc { get; set; }
62      public int DeviceId { get; set; }
63      public string DeviceName { get; set; } = "";
64      public string Location { get; set; } = "";
65      public int? CardholderId { get; set; }
66      public string? CardholderName { get; set; }
67      public EventKind Kind { get; set; }
68      public bool? Granted { get; set; }
69      public string? Detail { get; set; }
70
71      /// <summary>"Granted", "Denied" or the status change, for display.</summary>
72      public string Outcome => Granted switch
73      {
74          true => "Granted",
75          false => "Denied",
76          null => Detail ?? "",
77      };
78  }
```

Line 72 A switch on `bool?` covers all three cases, so the list and the web page show the same words without either deciding.

C# snack2-access-event-log/src/AccessLog.Core/Model/Model.cs, lines 98–113

```
98  /// <summary>Which events to list. Every filter is optional.</summary>
99  public sealed class EventQuery
100 {
101     public DateTime? FromUtc { get; set; }
102     public DateTime? ToUtc { get; set; }
103     public int? DeviceId { get; set; }
104     public int? CardholderId { get; set; }
105     public EventKind? Kind { get; set; }
106     public bool? Granted { get; set; }
107
108     /// <summary>Matches device name, location or cardholder name (case-insensitive).</summary>
109     public string? Text { get; set; }
110
111     /// <summary>Newest events first, at most this many.</summary>
112     public int Limit { get; set; } = 200;
113 }
```

One object carries every filter, all optional. Adding a filter later is a new property, not a new method on the interface, so neither implementation's callers break.

2.6 The repository interface

C# snack2-access-event-log/src/AccessLog.Core/Data/IAccessEventRepository.cs, lines 8–31

```
8   /// <summary>
9   /// Everything the app needs from storage. Two implementations: SQL Server (Dapper) for the real
10  /// thing, in-memory for tests and for running without a database. The same contract tests run
11  /// against both, so they behave the same.
12  /// </summary>
13  public interface IAccessEventRepository
14  {
15      Task<int> AddDeviceAsync(Device device);
16      Task<int> AddCardholderAsync(Cardholder cardholder);
17      Task<IReadOnlyList<Device>> GetDevicesAsync();
18      Task<IReadOnlyList<Cardholder>> GetCardholdersAsync();
19
20      /// <summary>Saves the event and returns its new id. The device (and cardholder, if any) mus
          t exist.</summary>
21      Task<long> RecordAsync(AccessEvent accessEvent);
22
```

```
23      /// <summary>Events matching the query, newest first, at most <see cref="EventQuery.Limit"/>
          .</summary>
24      Task<IReadOnlyList<AccessEventView>> SearchAsync(EventQuery query);
25
26      /// <summary>Denied badge swipes per door since a time, most denials first. Doors with none
          are left out.</summary>
27      Task<IReadOnlyList<DeniedByDoorRow>> DeniedByDoorAsync(DateTime sinceUtc);
28
29      /// <summary>The devices with the most events of any kind since a time, most first, at most
          <paramref name="top"/>.</summary>
30      Task<IReadOnlyList<NoisyDeviceRow>> NoisiestDevicesAsync(DateTime sinceUtc, int top);
31  }
```

Everything is `async`: database calls wait on the network, and an async method lets the UI thread keep drawing meanwhile. The comments are the contract: "newest first", "doors with none are left out", "most first". The contract tests check exactly these sentences.

C# snack2-access-event-log/src/AccessLog.Core/Data/IAccessEventRepository.cs, lines 33–34

```
33  /// <summary>A reference to something that doesn't exist, e.g. an event for an unknown device.</
        summary>
34  public sealed class UnknownReferenceException(string message) : Exception(message);
```

Both implementations translate their own failure (a SQL foreign-key error, a missing list entry) into this one exception, so callers handle one thing.

2.7 schema.sql

SQL snack2-access-event-log/src/AccessLog.Core/Data/schema.sql, lines 3–9

```
3   IF OBJECT_ID(N'dbo.Devices', N'U') IS NULL
4   CREATE TABLE dbo.Devices (
5       Id       INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Devices PRIMARY KEY,
6       Name     NVARCHAR(100) NOT NULL,
7       Kind     TINYINT       NOT NULL CONSTRAINT CK_Devices_Kind CHECK (Kind IN (0, 1)), -- 0 door
          , 1 camera
8       Location NVARCHAR(100) NOT NULL
9   );
```

Line 3 **Idempotent**: safe to run on every start. `OBJECT_ID` returns null if no user table (`'U'`) has that name.

Line 7 The enum's values, enforced. Naming constraints (`CK_...`) makes error messages and later changes readable.

SQL snack2-access-event-log/src/AccessLog.Core/Data/schema.sql, lines 18–30

```
18  IF OBJECT_ID(N'dbo.AccessEvents', N'U') IS NULL
19  CREATE TABLE dbo.AccessEvents (
20      Id            BIGINT IDENTITY(1,1) NOT NULL CONSTRAINT PK_AccessEvents PRIMARY KEY,
21      OccurredAtUtc DATETIME2(3)  NOT NULL,
22      DeviceId      INT           NOT NULL CONSTRAINT FK_AccessEvents_Devices REFERENCES dbo.Devic
          es (Id),
23      CardholderId  INT           NULL     CONSTRAINT FK_AccessEvents_Cardholders REFERENCES dbo.C
          ardholders (Id),
24      Kind          TINYINT       NOT NULL, -- 0 badge swipe, 1 status change
25      Granted       BIT           NULL,     -- null for status changes
26      Detail        NVARCHAR(200) NULL,
27      -- A badge swipe always has a cardholder and a decision; a status change has neither.
28      CONSTRAINT CK_AccessEvents_Shape CHECK (
29          (Kind = 0 AND CardholderId IS NOT NULL AND Granted IS NOT NULL) OR
```

```
30          (Kind = 1 AND CardholderId IS NULL AND Granted IS NULL))
```

Line 20 `BIGINT`: events accumulate forever; an `INT` runs out at about two billion.

Line 21 Millisecond precision, no time zone: the column name says it's UTC.

Line 28 The rule from Figure 2.1 as SQL. A badge swipe (Kind 0) must have a cardholder and a decision; a status change (Kind 1) must have neither.

SQL snack2-access-event-log/src/AccessLog.Core/Data/schema.sql, lines 33–41

```
33  -- Both reports filter on a time window and then group by device. Keyed on time, so "the last
34  -- 24 hours" is a range seek; DeviceId and Granted are included so the reports never touch the
35  -- table itself.
36  IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_AccessEvents_OccurredAt')
37  CREATE INDEX IX_AccessEvents_OccurredAt ON dbo.AccessEvents (OccurredAtUtc) INCLUDE (DeviceId, G
        ranted, Kind);
38
39  -- The event list filtered to one device, newest first.
40  IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_AccessEvents_Device_OccurredAt')
41  CREATE INDEX IX_AccessEvents_Device_OccurredAt ON dbo.AccessEvents (DeviceId, OccurredAtUtc DESC
        );
```

Line 37 Keyed on time, so a time range is a seek. The `INCLUDE` columns ride along in the index's leaf pages without being part of its order: enough to answer both reports from the index alone.

Line 41 For the event list filtered to one device, newest first: the index is already in the order the query wants.

2.8 SQL Server with Dapper

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 23–47

```
23      /// <summary>Creates the database if it's missing, then the tables and indexes (schema.sql).
          </summary>
24      public static async Task EnsureDatabaseAsync(string connectionString)
25      {
26          var builder = new SqlConnectionStringBuilder(connectionString);
27          var database = builder.InitialCatalog;
28          if (string.IsNullOrEmpty(database))
29              throw new ArgumentException("The connection string needs a database name (Database=.
          ..).");
30
31          // CREATE DATABASE has to run from another database, so connect to master first.
32          builder.InitialCatalog = "master";
33          using (var master = new SqlConnection(builder.ConnectionString))
34          {
35              // EXEC() only takes strings and variables, so build the statement first. QUOTENAME
36              // brackets the name, so it can't be used to inject SQL.
37              await master.ExecuteAsync(
38                  """
39                  DECLARE @create NVARCHAR(300) = N'CREATE DATABASE ' + QUOTENAME(@database);
40                  IF DB_ID(@database) IS NULL EXEC (@create);
41                  """,
42                  new { database });
43          }
44
45          using var connection = new SqlConnection(connectionString);
46          await connection.ExecuteAsync(ReadSchema());
47      }
```

Line 32 You can't create a database while connected to it, so this connects to `master` first.

Line 39 `EXEC()` accepts only strings and variables, not a function call, so the statement is built first. (The first version passed `QUOTENAME(...)` straight to `EXEC`; CI's SQL Server job failed every database test until it was fixed.)

Line 39 Brackets the name and escapes any `]` in it, so a database name can't smuggle SQL in. The name itself arrives as a parameter, never pasted into the string.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 99–125

```
 99      public async Task<long> RecordAsync(AccessEvent accessEvent)
100      {
101          using var connection = Open();
102          try
103          {
104              accessEvent.Id = await connection.ExecuteScalarAsync<long>(
105                  """
106                  INSERT INTO dbo.AccessEvents (OccurredAtUtc, DeviceId, CardholderId, Kind, Grant
    ed, Detail)
107                  OUTPUT INSERTED.Id
108                  VALUES (@OccurredAtUtc, @DeviceId, @CardholderId, @Kind, @Granted, @Detail);
109                  """,
110                  new
111                  {
112                      OccurredAtUtc = Utc(accessEvent.OccurredAtUtc),
113                      accessEvent.DeviceId,
114                      accessEvent.CardholderId,
115                      Kind = (byte)accessEvent.Kind,
116                      accessEvent.Granted,
117                      accessEvent.Detail,
118                  });
119          }
120          catch (SqlException e) when (e.Number == ForeignKeyViolation)
121          {
122              throw new UnknownReferenceException($"No device {accessEvent.DeviceId} or cardholder
    {accessEvent.CardholderId}.");
123          }
124          return accessEvent.Id;
125      }
```

Line 65 Returns the new identity value from the same statement: no second round trip, and no `SCOPE_IDENTITY()` pitfalls.

Line 115 The column is `TINYINT`; passing a `byte` makes the parameter's type match it exactly.

Line 120 An exception **filter**: only error 547 is caught and translated; anything else propagates untouched, stack trace intact.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 127–161

```
127      public async Task<IReadOnlyList<AccessEventView>> SearchAsync(EventQuery query)
128      {
129          // Each filter is "parameter is null OR condition", so one statement serves every combin
    ation.
130          // OPTION (RECOMPILE) lets SQL Server plan for the filters actually given each time.
131          const string sql = """
132              SELECT TOP (@Limit)
133                     e.Id, e.OccurredAtUtc, e.DeviceId, d.Name AS DeviceName, d.Location,
134                     e.CardholderId, c.Name AS CardholderName, e.Kind, e.Granted, e.Detail
135                FROM dbo.AccessEvents e
136                JOIN dbo.Devices d          ON d.Id = e.DeviceId
137                LEFT JOIN dbo.Cardholders c ON c.Id = e.CardholderId
138               WHERE (@FromUtc IS NULL      OR e.OccurredAtUtc >= @FromUtc)
```

```
139                AND (@ToUtc IS NULL        OR e.OccurredAtUtc <  @ToUtc)
140                AND (@DeviceId IS NULL     OR e.DeviceId = @DeviceId)
141                AND (@CardholderId IS NULL OR e.CardholderId = @CardholderId)
142                AND (@Kind IS NULL         OR e.Kind = @Kind)
143                AND (@Granted IS NULL      OR e.Granted = @Granted)
144                AND (@Text IS NULL         OR d.Name LIKE @Text OR d.Location LIKE @Text OR c.Nam
           e LIKE @Text)
145              ORDER BY e.OccurredAtUtc DESC, e.Id DESC
146             OPTION (RECOMPILE);
147             """;
148          using var connection = Open();
149          var rows = await connection.QueryAsync<AccessEventView>(sql, new
150          {
151              Limit = Math.Max(0, query.Limit),
152              FromUtc = query.FromUtc.HasValue ? Utc(query.FromUtc.Value) : (DateTime?)null,
153              ToUtc = query.ToUtc.HasValue ? Utc(query.ToUtc.Value) : (DateTime?)null,
154              query.DeviceId,
155              query.CardholderId,
156              Kind = query.Kind.HasValue ? (byte)query.Kind.Value : (byte?)null,
157              query.Granted,
158              Text = string.IsNullOrWhiteSpace(query.Text) ? null : "%" + EscapeLike(query.Text!.T
           rim()) + "%",
159          });
160          return rows.Select(AsUtc).ToList();
161      }
```

Line 138 The "optional filter" pattern: when a parameter is null the condition is always true. One statement serves every combination of filters.

Line 130 Without it, SQL Server caches one plan for this statement, made for whichever filters the first call used, and reuses it for very different ones. Recompiling costs a little CPU per call and gets a plan for the actual values.

Line 158 The pattern is built in C# and sent as a parameter, still never pasted into the SQL.

Line 160 `DATETIME2` has no time zone, so values come back as `DateTimeKind.Unspecified`; they're marked UTC before anyone can mistake them for local times.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 163–180

```
163      public async Task<IReadOnlyList<DeniedByDoorRow>> DeniedByDoorAsync(DateTime sinceUtc)
164      {
165          // GROUP BY the door; only denied swipes in the window count. The time-keyed index makes
           the
166          // WHERE a range seek, and it includes DeviceId and Granted, so the table itself isn't r
           ead.
167          const string sql = """
168              SELECT d.Id AS DeviceId, d.Name AS DeviceName, d.Location, COUNT(*) AS Denied
169                FROM dbo.AccessEvents e
170                JOIN dbo.Devices d ON d.Id = e.DeviceId
171               WHERE e.OccurredAtUtc >= @sinceUtc
172                 AND e.Kind = 0
173                 AND e.Granted = 0
174               GROUP BY d.Id, d.Name, d.Location
175               ORDER BY Denied DESC, d.Name;
176              """;
177          using var connection = Open();
178          var rows = await connection.QueryAsync<DeniedByDoorRow>(sql, new { sinceUtc = Utc(sinceU
           tc) });
179          return rows.ToList();
180      }
```

The first report, as SQL you could paste into SQL Server Management Studio. `WHERE` narrows to denied badge swipes in the window (a range seek on the time index); `GROUP BY` makes one row

per door; `COUNT(*)` counts each group. Every non-aggregated column in the `SELECT` is in the `GROUP BY`, which SQL requires. The ordering tie-break on name makes the result deterministic.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 182–195

```
182     public async Task<IReadOnlyList<NoisyDeviceRow>> NoisiestDevicesAsync(DateTime sinceUtc, int
           top)
183     {
184         const string sql = """
185             SELECT TOP (@top) d.Id AS DeviceId, d.Name AS DeviceName, d.Kind, COUNT(*) AS Events
186               FROM dbo.AccessEvents e
187               JOIN dbo.Devices d ON d.Id = e.DeviceId
188              WHERE e.OccurredAtUtc >= @sinceUtc
189              GROUP BY d.Id, d.Name, d.Kind
190              ORDER BY Events DESC, d.Name;
191             """;
192         using var connection = Open();
193         var rows = await connection.QueryAsync<NoisyDeviceRow>(sql, new { sinceUtc = Utc(sinceUt
          c), top = Math.Max(0, top) });
194         return rows.ToList();
195     }
```

`TOP (@top)` with parentheses accepts a parameter. Every kind of event counts here, so a flapping camera shows up as noisy, which is the point of the report.

C# snack2-access-event-log/src/AccessLog.Core/Data/SqlServerAccessEventRepository.cs, lines 207–209

```
207     /// <summary>Makes %, _ and [ in user text match themselves in a LIKE pattern.</summary>
208     private static string EscapeLike(string text) =>
209         text.Replace("[", "[[]").Replace("%", "[%]").Replace("_", "[_]");
```

In a `LIKE` pattern, `%` and _ are wildcards. Wrapping them in brackets makes them literal, so searching for "_" finds underscores instead of everything. A contract test checks it.

2.9 The in-memory twin

C# snack2-access-event-log/src/AccessLog.Core/Data/InMemoryAccessEventRepository.cs, lines 55–77

```
55      public Task<long> RecordAsync(AccessEvent accessEvent)
56      {
57          lock (_gate)
58          {
59              if (_devices.All(d => d.Id != accessEvent.DeviceId) ||
60                  (accessEvent.CardholderId is int c && _cardholders.All(x => x.Id != c)))
61                  throw new UnknownReferenceException($"No device {accessEvent.DeviceId} or cardho
            lder {accessEvent.CardholderId}.");
62              CheckShape(accessEvent);
63
64              accessEvent.Id = _nextEventId++;
65              _events.Add(new AccessEvent
66              {
67                  Id = accessEvent.Id,
68                  OccurredAtUtc = Utc(accessEvent.OccurredAtUtc),
69                  DeviceId = accessEvent.DeviceId,
70                  CardholderId = accessEvent.CardholderId,
71                  Kind = accessEvent.Kind,
72                  Granted = accessEvent.Granted,
73                  Detail = accessEvent.Detail,
74              });
75              return Task.FromResult(accessEvent.Id);
76          }
77      }
```

Line 61 The foreign keys, in code: same exception as SQL Server's translation.

Line 62 The `CHECK` constraint, in code.

Line 68 Stored the way SQL Server would store it: UTC, to the millisecond (see `Utc` at the bottom of the file).

C# snack2-access-event-log/src/AccessLog.Core/Data/InMemoryAccessEventRepository.cs, lines 79–97

```
79      public Task<IReadOnlyList<AccessEventView>> SearchAsync(EventQuery query)
80      {
81          lock (_gate)
82          {
83              var text = string.IsNullOrWhiteSpace(query.Text) ? null : query.Text!.Trim();
84              var rows = Views()
85                  .Where(e => query.FromUtc is not DateTime start || e.OccurredAtUtc >= Utc(start)
          )
86                  .Where(e => query.ToUtc is not DateTime end || e.OccurredAtUtc < Utc(end))
87                  .Where(e => query.DeviceId is null || e.DeviceId == query.DeviceId)
88                  .Where(e => query.CardholderId is null || e.CardholderId == query.CardholderId)
89                  .Where(e => query.Kind is null || e.Kind == query.Kind)
90                  .Where(e => query.Granted is null || e.Granted == query.Granted)
91                  .Where(e => text is null || Contains(e.DeviceName, text) || Contains(e.Location,
             text) || Contains(e.CardholderName, text))
92                  .OrderByDescending(e => e.OccurredAtUtc).ThenByDescending(e => e.Id)
93                  .Take(Math.Max(0, query.Limit))
94                  .ToList();
95              return Task.FromResult<IReadOnlyList<AccessEventView>>(rows);
96          }
97      }
```

The same filters as the SQL, in LINQ, with the same ordering (time, then id, both descending). The text match uses `IndexOf(..., OrdinalIgnoreCase)` because .NET Framework has no case-insensitive `Contains`; SQL Server's default collation is case-insensitive too.

C# snack2-access-event-log/src/AccessLog.Core/Data/InMemoryAccessEventRepository.cs, lines 144–161

```
144     private IEnumerable<AccessEventView> Views() =>
145         from e in _events
146         join d in _devices on e.DeviceId equals d.Id
147         join c in _cardholders on e.CardholderId equals (int?)c.Id into cs
148         from c in cs.DefaultIfEmpty()
149         select new AccessEventView
150         {
151             Id = e.Id,
152             OccurredAtUtc = e.OccurredAtUtc,
153             DeviceId = d.Id,
154             DeviceName = d.Name,
155             Location = d.Location,
156             CardholderId = e.CardholderId,
157             CardholderName = c?.Name,
158             Kind = e.Kind,
159             Granted = e.Granted,
160             Detail = e.Detail,
161         };
```

A LINQ **left join**: `join ... into cs` groups the matching cardholders, and `DefaultIfEmpty()` yields a single null when there are none, so status changes survive the join. The `(int?)c.Id` cast is needed because `CardholderId` is nullable and the two sides of `equals` must have the same type.

2.10 The simulator

C# snack2-access-event-log/src/AccessLog.Core/Simulation/AccessSimulator.cs, lines 10–14

```
10  /// <summary>The current time, injectable so tests can fix it.</summary>
11  public interface IClock
12  {
13      DateTime UtcNow { get; }
14  }
```

The current time behind an interface, so tests can freeze it. Code that calls `DateTime.UtcNow` directly can't be tested at "exactly 24 hours ago".

C# snack2-access-event-log/src/AccessLog.Core/Simulation/AccessSimulator.cs, lines 73–88

```
73      private AccessEvent Swipe(DateTime atUtc)
74      {
75          // Some doors are busier than others: pick from the first doors more often.
76          var door = _doors[Math.Min(random.Next(_doors.Count) , random.Next(_doors.Count))];
77          var cardholder = _cardholders[random.Next(_cardholders.Count)];
78          var granted = !RevokedBadges.Contains(cardholder.BadgeNumber) && random.Next(100) < 92;
79          return new AccessEvent
80          {
81              OccurredAtUtc = atUtc,
82              DeviceId = door.Id,
83              CardholderId = cardholder.Id,
84              Kind = EventKind.BadgeSwipe,
85              Granted = granted,
86              Detail = granted ? null : "Badge refused",
87          };
88      }
```

Line 76 The smaller of two random picks favours the first doors, so the noisiest-device report has a clear winner instead of a tie.

Line 78 Two badges are always refused, so the denied report always has something to show; a test checks they're never granted.

C# snack2-access-event-log/src/AccessLog.Core/Simulation/AccessSimulator.cs, lines 90–104

```
90      private AccessEvent StatusChange(DateTime atUtc)
91      {
92          var camera = _cameras[random.Next(_cameras.Count)];
93          var from = _cameraStatus.TryGetValue(camera.Id, out var current) ? current : "Online";
94          var options = new[] { "Online", "Offline", "Alarm" }.Where(s => s != from).ToArray();
95          var to = options[random.Next(options.Length)];
96          _cameraStatus[camera.Id] = to;
97          return new AccessEvent
98          {
99              OccurredAtUtc = atUtc,
100             DeviceId = camera.Id,
101             Kind = EventKind.StatusChange,
102             Detail = $"{from} → {to}",
103         };
104     }
```

Each camera remembers its last status, so the history reads like Snack 1's: "Online → Alarm", then "Alarm → Offline", never two "from" values that don't follow on. A test walks every camera's history to check it.

2.11 The ViewModel

C# snack2-access-event-log/src/AccessLog.Core/Mvvm/Mvvm.cs, lines 27–56

```
27  /// <summary>
28  /// A command that runs async work. It disables itself while running, so a double-click can't
29  /// start the same database query twice. <see cref="ExecuteAsync"/> lets tests await the work.
30  /// </summary>
31  public sealed class AsyncRelayCommand(Func<Task> execute) : ICommand
32  {
33      private bool _running;
34
35      public event EventHandler? CanExecuteChanged;
36
37      public bool CanExecute(object? parameter) => !_running;
38
39      public async void Execute(object? parameter) => await ExecuteAsync();
40
41      public async Task ExecuteAsync()
42      {
43          if (_running) return;
44          _running = true;
45          CanExecuteChanged?.Invoke(this, EventArgs.Empty);
46          try
47          {
48              await execute();
49          }
50          finally
51          {
52              _running = false;
53              CanExecuteChanged?.Invoke(this, EventArgs.Empty);
54          }
55      }
56  }
```

Line 37 The button disables itself while its query runs, so a double-click can't send the same query twice.

Line 39 `async void` is only acceptable for event-handler-like entry points such as `ICommand.Execute`, which must return void. Tests call `ExecuteAsync` and await it.

C# snack2-access-event-log/src/AccessLog.Core/ViewModels/EventLogViewModel.cs, lines 101–117

```
101     /// <summary>The query the current filters describe.</summary>
102     public EventQuery CurrentQuery() => new()
103     {
104         Text = string.IsNullOrWhiteSpace(SearchText) ? null : SearchText.Trim(),
105         Kind = Outcome switch
106         {
107             OutcomeFilter.Granted or OutcomeFilter.Denied => EventKind.BadgeSwipe,
108             OutcomeFilter.StatusChanges => EventKind.StatusChange,
109             _ => null,
110         },
111         Granted = Outcome switch
112         {
113             OutcomeFilter.Granted => true,
114             OutcomeFilter.Denied => false,
115             _ => null,
116         },
117         Limit = 200,
```

The filters as a query object, separate from running it, so a test can check the translation without a database: "Denied" means badge swipes with `Granted = false`.

C# snack2-access-event-log/src/AccessLog.Core/ViewModels/EventLogViewModel.cs, lines 120–133

```
120     /// <summary>
121     /// Reloads the list and both reports. If a refresh is already running, this one waits for i
          t
122     /// and then runs once more, so the last filter change always wins and queries never overlap
          .
123     /// </summary>
124     public Task RefreshAsync()
125     {
126         if (_refreshing is { IsCompleted: false })
127         {
128             _refreshAgain = true;
129             return _refreshing;
130         }
131         _refreshing = RefreshLoopAsync();
132         return _refreshing;
133     }
```

C# snack2-access-event-log/src/AccessLog.Core/ViewModels/EventLogViewModel.cs, lines 135–158

```
135     private async Task RefreshLoopAsync()
136     {
137         do
138         {
139             _refreshAgain = false;
140             try
141             {
142                 var since = _clock.UtcNow.AddHours(-ReportHours);
143                 var events = await _repository.SearchAsync(CurrentQuery());
144                 var denied = await _repository.DeniedByDoorAsync(since);
145                 var noisy = await _repository.NoisiestDevicesAsync(since, TopDevices);
146 
147                 Replace(Events, events);
148                 Replace(DeniedByDoor, denied);
149                 Replace(NoisiestDevices, noisy);
150                 LastRefreshedUtc = _clock.UtcNow;
151                 Status = $"{events.Count} events shown · {denied.Sum(d => d.Denied)} denied in t
              he last {ReportHours} h";
152             }
153             catch (Exception e)
154             {
155                 Status = "Couldn't load events: " + e.Message;
156             }
157         } while (_refreshAgain);
158     }
```

Line 126 Typing "server" sets the search five times in a row. Rather than run five overlapping queries (whose answers could arrive out of order, leaving the list showing an older filter), a refresh that arrives mid-query just asks for one more pass.

Line 157 The loop runs again if anything asked while it was busy, so the last filter always wins and queries never overlap.

Line 155 A database outage becomes a message in the status bar, not a crashed app. A test with a broken repository checks it.

2.12 The .NET Framework 4.8 app

MSBuild snack2-access-event-log/src/AccessLog.App/AccessLog.App.csproj, lines 1–19

```
<Project Sdk="Microsoft.NET.Sdk">

  <!-- The WPF app on .NET Framework 4.8, as the README asks. Everything it shows comes from AccessLog.Core. -->
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net48</TargetFramework>
    <UseWPF>true</UseWPF>
    <AssemblyName>AccessLog</AssemblyName>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.NETFramework.ReferenceAssemblies" Version="1.0.3" PrivateAssets="all" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\AccessLog.Core\AccessLog.Core.csproj" />
  </ItemGroup>

</Project>
```

An SDK-style project targeting `net48`: the modern project format, built by the .NET 10 SDK, producing an app for .NET Framework 4.8. No `ImplicitUsings` here: on .NET Framework they include `System.Net.Http`, which isn't referenced by default, and the build fails.

C# snack2-access-event-log/src/AccessLog.App/Services/ViewModelLocator.cs, lines 24–48

```
    private static EventLogViewModel Create()
    {
        var clock = new SystemClock();
        // Startup work runs on a pool thread and is waited for here, so the async database
        // calls never need the UI thread (waiting on it from the UI thread would deadlock).
        var (repository, description) = Task.Run(() => ChooseStoreAsync()).GetAwaiter().GetResult();
        var simulator = new AccessSimulator(repository, new Random(), clock);
        Task.Run(async () =>
        {
            await simulator.SeedAsync();
            if ((await repository.SearchAsync(new Core.Model.EventQuery { Limit = 1 })).Count == 0)
                await simulator.BackfillAsync(hours: 24, events: 400);
        }).GetAwaiter().GetResult();

        var viewModel = new EventLogViewModel(repository, clock, description);
        var timer = new DispatcherTimer { Interval = TimeSpan.FromSeconds(1.5) };
        timer.Tick += async (_, _) =>
        {
            await simulator.StepAsync();
            await viewModel.RefreshAsync();
        };
        timer.Start();
        _ = viewModel.RefreshAsync();
        return viewModel;
    }
```

Line 29 Waiting on async code from the UI thread deadlocks if the async code needs the UI thread to continue (it captures the UI's synchronization context). `Task.Run` moves the work to a pool thread, which has no such context, so waiting is safe. It's

acceptable once, at startup; never in an event handler.

Line 39 A `DispatcherTimer` ticks on the UI thread, so its handler can update bound collections directly; no dispatcher hop needed, unlike Snack 1's thread-pool timer.

C# snack2-access-event-log/src/AccessLog.App/Services/ViewModelLocator.cs, lines 50–64

```
50      /// <summary>SQL Server if it answers within a few seconds, otherwise memory (and say so).</
          summary>
51      private static async Task<(IAccessEventRepository, string)> ChooseStoreAsync()
52      {
53          var connection = Environment.GetEnvironmentVariable("ACCESSLOG_CONNECTION") ?? DefaultCo
          nnection;
54          try
55          {
56              await SqlServerAccessEventRepository.EnsureDatabaseAsync(connection);
57              var builder = new Microsoft.Data.SqlClient.SqlConnectionStringBuilder(connection);
58              return (new SqlServerAccessEventRepository(connection), $"SQL Server: {builder.Initi
          alCatalog} on {builder.DataSource}");
59          }
60          catch (Exception e)
61          {
62              return (new InMemoryAccessEventRepository(), $"In memory (SQL Server not reachable:
          {e.Message})");
63          }
64      }
```

SQL Server if it answers (the connection string has `Connect Timeout=3`), otherwise memory, and the status bar says which and why. A demo that starts on any machine is worth more than one that crashes without Docker.

XAML snack2-access-event-log/src/AccessLog.App/MainWindow.xaml, lines 56–89

```
56              <!-- Reports -->
57              <TabItem Header="Reports">
58                  <Grid Margin="8">
59                      <Grid.ColumnDefinitions>
60                          <ColumnDefinition Width="*" />
61                          <ColumnDefinition Width="16" />
62                          <ColumnDefinition Width="*" />
63                      </Grid.ColumnDefinitions>
64                      <DockPanel>
65                          <TextBlock DockPanel.Dock="Top" Text="Denied access by door, last 24 h"
66                                     FontWeight="SemiBold" Margin="0,0,0,8" />
67                          <DataGrid ItemsSource="{Binding DeniedByDoor}" AutoGenerateColumns="Fals
          e" IsReadOnly="True"
68                                    HeadersVisibility="Column">
69                              <DataGrid.Columns>
70                                  <DataGridTextColumn Header="Door" Width="*" Binding="{Binding De
          viceName}" />
71                                  <DataGridTextColumn Header="Location" Width="*" Binding="{Bindin
          g Location}" />
72                                  <DataGridTextColumn Header="Denied" Width="80" Binding="{Binding
           Denied}" />
73                              </DataGrid.Columns>
74                          </DataGrid>
75                      </DockPanel>
76                      <DockPanel Grid.Column="2">
77                          <TextBlock DockPanel.Dock="Top" Text="Top 5 noisiest devices, last 24 h"
78                                     FontWeight="SemiBold" Margin="0,0,0,8" />
79                          <DataGrid ItemsSource="{Binding NoisiestDevices}" AutoGenerateColumns="F
          alse" IsReadOnly="True"
80                                    HeadersVisibility="Column">
81                              <DataGrid.Columns>
82                                  <DataGridTextColumn Header="Device" Width="*" Binding="{Binding
          DeviceName}" />
```

```
                                <DataGridTextColumn Header="Kind" Width="90" Binding="{Binding K
        ind}" />
                                <DataGridTextColumn Header="Events" Width="80" Binding="{Binding
         Events}" />
                            </DataGrid.Columns>
                        </DataGrid>
                    </DockPanel>
                </Grid>
            </TabItem>
```

Two `DataGrid`s side by side, each bound to one report collection. `AutoGenerateColumns="False"` with explicit columns keeps the headers human and the order fixed.

2.13 The contract tests

C# snack2-access-event-log/tests/AccessLog.Tests/RepositoryContract.cs, lines 6–39

```
/// <summary>
/// The behaviour every IAccessEventRepository must have. InMemoryRepositoryTests and
/// SqlServerRepositoryTests inherit these tests, so both stores are held to exactly the same ru
    les,
/// and the in-memory one can stand in for SQL Server everywhere else.
/// </summary>
public abstract class RepositoryContract : IAsyncLifetime
{
    // Whole seconds: SQL Server keeps milliseconds (DATETIME2(3)), so finer times wouldn't roun
      d-trip.
    protected static readonly DateTime Now = new(2026, 11, 12, 15, 0, 0, DateTimeKind.Utc);

    protected IAccessEventRepository Repo { get; private set; } = null!;
    protected Device MainDoor { get; private set; } = null!;
    protected Device ServerRoom { get; private set; } = null!;
    protected Device Lobby { get; private set; } = null!;
    protected Cardholder Alice { get; private set; } = null!;
    protected Cardholder Bob { get; private set; } = null!;

    /// <summary>A new, empty store for each test.</summary>
    protected abstract Task<IAccessEventRepository> CreateAsync();

    public virtual async ValueTask InitializeAsync()
    {
        Repo = await CreateAsync();
        MainDoor = new Device { Name = "Main entrance", Kind = DeviceKind.Door, Location = "Grou
      nd floor" };
        ServerRoom = new Device { Name = "Server room", Kind = DeviceKind.Door, Location = "Floo
      r 2" };
        Lobby = new Device { Name = "Lobby camera", Kind = DeviceKind.Camera, Location = "Ground
       floor" };
        foreach (var d in new[] { MainDoor, ServerRoom, Lobby }) await Repo.AddDeviceAsync(d);
        Alice = new Cardholder { Name = "Alice Martin", BadgeNumber = "B-1" };
        Bob = new Cardholder { Name = "Bob Tremblay", BadgeNumber = "B-2" };
        await Repo.AddCardholderAsync(Alice);
        await Repo.AddCardholderAsync(Bob);
    }

    public virtual ValueTask DisposeAsync() => ValueTask.CompletedTask;
```

Line 14 Whole seconds on a fixed date: SQL Server keeps milliseconds, so finer times wouldn't come back equal, and a fixed date makes failures reproducible.

Line 24 The only thing a subclass provides: a fresh, empty store. Every `[Fact]` below runs once per subclass.

C# snack2-access-event-log/tests/AccessLog.Tests/RepositoryContract.cs, lines 180–193

```
180     public async Task Denied_by_door_counts_only_denied_swipes_in_the_window_most_first()
181     {
182         await Swipe(ServerRoom, Alice, false, Now.AddHours(-1));
183         await Swipe(ServerRoom, Bob, false, Now.AddHours(-2));
184         await Swipe(MainDoor, Bob, false, Now.AddHours(-3));
185         await Swipe(MainDoor, Alice, true, Now.AddHours(-1));          // granted: doesn't count
186         await Swipe(MainDoor, Alice, false, Now.AddHours(-25));        // too old: doesn't count
187         await StatusChange(Lobby, "Online → Alarm", Now.AddHours(-1)); // not a swipe
188
189         var rows = await Repo.DeniedByDoorAsync(Now.AddHours(-24));
190
191         Assert.Equal(new[] { ("Server room", 2), ("Main entrance", 1) }, rows.Select(r => (r.Dev
          iceName, r.Denied)));
192         Assert.Equal("Floor 2", rows[0].Location);
193     }
```

Each line of setup is there to be excluded for a reason the comment gives: granted, too old, not a swipe. If the SQL's `WHERE` or the LINQ's `Where` forgets one condition, this test fails for that store.

C# snack2-access-event-log/tests/AccessLog.Tests/SqlServerRepositoryTests.cs, lines 20–24

```
20      public override ValueTask InitializeAsync()
21      {
22          Assert.SkipWhen(Server is null, "Set TEST_SQLSERVER to a SQL Server connection string (n
          o database name) to run these.");
23          return base.InitializeAsync();
24      }
```

C# snack2-access-event-log/tests/AccessLog.Tests/SqlServerRepositoryTests.cs, lines 63–78

```
63      public async Task The_reports_are_answered_from_the_time_index()
64      {
65          // Ask SQL Server for the estimated plan of the "denied by door" query and check it uses
66          // the index the schema creates for exactly this purpose.
67          await using var connection = new SqlConnection(ConnectionString);
68          await connection.OpenAsync(TestContext.Current.CancellationToken);
69          await connection.ExecuteAsync("SET SHOWPLAN_XML ON;");
70          var plan = await connection.ExecuteScalarAsync<string>("""
71              SELECT d.Id, COUNT(*) FROM dbo.AccessEvents e JOIN dbo.Devices d ON d.Id = e.DeviceI
          d
72               WHERE e.OccurredAtUtc >= DATEADD(HOUR, -24, SYSUTCDATETIME()) AND e.Kind = 0 AND e.
          Granted = 0
73               GROUP BY d.Id;
74              """);
75          await connection.ExecuteAsync("SET SHOWPLAN_XML OFF;");
76
77          Assert.Contains("IX_AccessEvents_OccurredAt", plan);
78      }
```

A test of a performance decision. `SET SHOWPLAN_XML ON` makes SQL Server return its execution plan instead of running the query; the test checks the plan names the index. If someone drops the index or rewrites the report so it can't use it, this fails.

Class	Runs	What it adds
InMemoryRepositoryTests	19 contract tests + 1	the shape rule in code
SqlServerRepositoryTests	19 contract tests + 3	the schema re-runs cleanly, the CHECK constraint, the plan uses the index

Class	Runs	What it adds
EventLogViewModelTests	9	refresh, filters, clear, overlapping refreshes, a failing store
AccessSimulatorTests	6	seeding once, valid events, revoked badges, status sequences, backfill

2.14 Interview questions

Why put a CHECK constraint in the database when the code already validates?

The database is shared by every client and every future version of this one. A constraint makes invalid rows impossible regardless of who writes them; validation in code only protects the code that runs it.

What's a covering index, and why does the report need one?

An index containing every column a query uses, either as keys or as INCLUDE columns, so the query is answered from the index without reading the table. The report filters on time and needs DeviceId, Kind and Granted; keying on time makes the window a seek, and including the rest makes it covering.

What does OPTION (RECOMPILE) fix in the search query?

Parameter sniffing: SQL Server caches one plan per statement, made for the first parameter values it saw. With "optional filter" predicates, a plan made for "no filters" is terrible for "one device", and vice versa. Recompiling makes a plan for the actual values each time.

How do you stop SQL injection with a search box?

Never build SQL from input: the text goes in as a parameter. The LIKE pattern is assembled in C# (with % and _ escaped) and passed as a parameter too. The database name in CREATE DATABASE is passed as a parameter and bracketed with QUOTENAME.

Why store UTC?

Local time is ambiguous (the hour repeated when clocks go back) and differs between sites. UTC is one timeline; convert to local only for display.

Why Dapper and not Entity Framework here?

The reports are the deliverable, so readable SQL matters, and EF Core doesn't support .NET Framework. Dapper does the tedious part (parameters, mapping) and nothing else.

How do you know the in-memory repository behaves like SQL Server?

The same contract tests run against both. Any difference in ordering, filtering or reporting fails a test for one of them.

2.15 Exercises

Paging

Add "load more" to the event list without OFFSET scanning millions of rows.

Hint. Keyset paging: pass the last row's (OccurredAtUtc, Id) and ask for rows before it; the existing ordering and indexes make that a seek. Add it to the contract.

Retention

Delete events older than 90 days without blocking the app.

Hint. Delete in batches of a few thousand in a loop (`DELETE TOP (5000) ... WHERE OccurredAtUtc < @cutoff`); the time index makes each batch cheap.

Denied by hour

A third report: denials per hour of the day over the last week.

Hint. `DATEPART(HOUR, OccurredAtUtc)` in the GROUP BY; decide whether the hours are UTC or local and say so in the UI.

Migrations

Add a column (the door's zone) to a database that already has data.

Hint. Write schema changes as numbered scripts and record which ran in a table; or adopt a tool such as DbUp.